

RepForge

Dokumentacja funkcjonalna, projektowa i techniczna

Aplikacja wspierająca planowanie treningów, prowadzenie aktywnej sesji, analizę postępów oraz podstawową ocenę techniki ćwiczeń na podstawie nagrania wideo.

Zakres dokumentu

Opis celu i zakresu projektu, wymagania funkcjonalne i нефункционалне, przypadki użycia, architektura, wybrane trudniejsze elementy implementacji oraz instrukcja użytkownika i uruchomienia.

Cel, problem i kryteria realizacji

Cel projektu

Celem RepForge jest połączenie planowania treningu, prowadzenia aktywnej sesji i późniejszej analizy wyników w jednym, spójnym narzędziu działającym na komputerze i telefonie.

Problem do rozwiązania

Najważniejszym problemem jest utrzymanie ciągłości treningu mimo zmiany widoku, odświeżenia strony lub pracy na urządzeniu mobilnym. Drugim problemem jest bezpieczne uruchamianie analizy nagrania bez blokowania pozostałych funkcji aplikacji.

Zakres

Projekt obejmuje konto użytkownika, plany treningowe, aktywną sesję, historię, statystyki, profil oraz wspomagającą analizę wideo. Nie obejmuje panelu administracyjnego, płatności ani diagnozy medycznej.

Kryteria realizacji

Kryterium	Sposób sprawdzenia
Użytkownik może utworzyć plan i ukończyć sesję.	Pełny scenariusz od formularza do historii treningów.
Aktywna sesja nie traci danych po opuszczeniu widoku.	Powrót do tego samego kroku, czasu, wpisów i notatek.
Statystyki wynikają z ukończonych treningów.	Porównanie danych historii z panelem postępu.
Dane użytkowników są od siebie oddzielone.	Próba odczytu zasobu należącego do innego konta.
Błąd analizy nagrania nie zatrzymuje aplikacji.	Kontrolowany status i możliwość dalszego korzystania z treningu.
Podstawowe działania są dostępne na telefonie.	Sprawdzenie formularzy, sesji i nawigacji na małym ekranie.

Ujednolicona terminologia

Nazwa używana w dokumencie	Znaczenie
Tryb demonstracyjny	Dane lokalne w przeglądarce; odpowiednik technicznego trybu mock.
Tryb z API	Frontend połączony z backendem ASP.NET Core i bazą PostgreSQL.
Model widoku	Logika przygotowująca dane i akcje dla komponentu interfejsu (view model).
Licznik czasu	Element odmierzający czas ćwiczenia, odpoczynku lub całej sesji.
Wersja mobilna	Układ przeglądarkowy na telefonie albo aplikacja Android przez Capacitor.

1. Charakterystyka i zakres projektu

RepForge jest aplikacją treningową łączącą planowanie, wykonanie i analizę treningu w jednym procesie. Użytkownik może przygotować plan ćwiczeń, określić liczbę serii, powtórzeń, obciążenie i czas odpoczynku, a następnie przeprowadzić sesję z licznikiem czasu i zapisem faktycznie wykonanych wyników.

Projekt obejmuje warstwę webową wykonaną w Next.js i React, backend ASP.NET Core z bazą PostgreSQL oraz moduł analizy nagrań napisany w Pythonie. Ten sam frontend może pracować w przeglądarce na komputerze, w układzie mobilnym oraz jako aplikacja Android oparta na Capacitorze.

Istotną cechą rozwiązania jest rozdzielenie interfejsu od źródła danych. Warstwa prezentacji korzysta z tych samych kontraktów zarówno w trybie demonstracyjnym, jak i podczas komunikacji z rzeczywistym API. Pozwala to rozwijać interfejs bez stale uruchomionego backendu, a następnie zweryfikować trwałość danych i autoryzację w pełnym środowisku.

Układ dokumentu

Część	Zawartość
Analiza	Cel, zakres, aktorzy, wymagania oraz przypadki użycia.
Projekt	Architektura, przepływ danych i podział odpowiedzialności.
Implementacja	Wybrane fragmenty kodu, które wymagały obsługi stanu, czasu, procesów zewnętrznych i responsywności.
Instrukcja	Opis ekranów, sposobu pracy użytkownika oraz uruchamiania projektu.

Główny aktor

Osoba trenująca, która tworzy plany, wykonuje sesje i analizuje własne wyniki. Administracja systemem nie jest osobnym modułem użytkowym projektu.

2. Wymagania funkcjonalne

Wymagania funkcjonalne opisują operacje dostępne z perspektywy użytkownika. Każde z nich jest powiązane z konkretnym modułem aplikacji i może zostać sprawdzone przez wykonanie pełnego scenariusza.

ID	Wymaganie	Sposób realizacji
WF-01	Rejestracja i logowanie użytkownika.	Konto, token JWT oraz izolacja danych właściciela.
WF-02	Tworzenie, edycja i usuwanie planu treningowego.	Formularz metadanych i ćwiczeń z walidacją.
WF-03	Planowanie daty i godziny treningu.	Pola harmonogramu oraz lista treningów oczekujących.
WF-04	Uruchomienie sesji na podstawie planu.	Generowanie kolejnych etapów ćwiczenia i odpoczynku.
WF-05	Zapis faktycznej liczby powtórzeń i realizacji celu.	Wpisy sesji przypisane do indeksu kroku.
WF-06	Wstrzymanie, wznowienie i kontynuowanie aktywnej sesji.	Stan sesji zapisany w API i odtwarzany po powrocie.
WF-07	Zakończenie sesji i utworzenie podsumowania.	Potwierdzenie operacji i zapis historii wykonania.
WF-08	Prezentacja aktywnego treningu poza ekranem sesji.	Pływający licznik z możliwością powrotu do treningu.
WF-09	Wyświetlanie statystyk, rekordów i serii regularności.	Agregacja ukończonych treningów po stronie backendu.
WF-10	Obsługa profilu i zmiany hasła.	Osobne operacje profilu z kontrolą bieżącego użytkownika.
WF-11	Przesłanie nagrania ćwiczenia do analizy.	Walidacja pliku, zapis rekordu i uruchomienie modułu Python.
WF-12	Prezentacja historii oraz wyniku analiz wideo.	Status, podsumowanie, metryki i dostęp do nagrania źródłowego lub przetworzonego.

3. Wymagania нефункционалне

Wymagania нефункционалне określają jakość działania systemu. W projekcie największe znaczenie mają ciągłość aktywnej sesji, izolacja danych użytkowników, responsywność interfejsu oraz kontrola kosztownych operacji związanych z analizą wideo.

ID	Obszar	Wymaganie / kryterium
WN-01	Bezpieczeństwo	Dane treningowe i analizy są dostępne wyłącznie dla uwierzytelnionego właściciela.
WN-02	Bezpieczeństwo	Sekret JWT i konfiguracja CORS nie mogą używać wartości deweloperskich w środowisku produkcyjnym.
WN-03	Niezawodność	Aktywna sesja nie może zniknąć po zmianie zakładki, odświeżeniu strony ani krótkiej utracie widoku.
WN-04	Spójność	Zapis postępu ma być idempotentny dla danego kroku sesji i nie może tworzyć duplikatów wpisów.
WN-05	Responsywność	Najważniejsze działania muszą być dostępne na komputerze i telefonie bez nachodzenia na nawigację systemową.
WN-06	Wydajność	Listy historii i analiz są ograniczane lub stronicowane, aby nie pobierać całego zbioru danych jednocześnie.
WN-07	Odporność	Błąd procesu Python, brak modelu albo przekroczenie czasu nie może przerwać działania API.
WN-08	Kontrola zasobów	Nagrania mają limit rozmiaru, liczbę analiz na użytkownika i określony czas przechowywania.
WN-09	Użyteczność	Błędy techniczne są mapowane na komunikaty zrozumiałe dla użytkownika.
WN-10	Testowalność	Logika domenowa i view modele mają testy automatyczne obejmujące także ciężkie przypadki.

Priorytet

Ciągłość sesji treningowej ma wyższy priorytet niż idealna zgodność timera z momentem odświeżenia interfejsu. Dlatego czas jest odtwarzany z danych zapisanych w API i znacznika ostatniej aktualizacji.

4. Przypadki użycia

Przypadki użycia przedstawiają typowe cele użytkownika i reakcję systemu. Uwzględniono zarówno przebieg podstawowy, jak i sytuacje, które wymagają walidacji albo odtworzenia stanu.

PU-01: Utworzenie planu treningowego

Element	Opis
Aktor	Uwierzytelniony użytkownik.
Warunek początkowy	Użytkownik znajduje się w module treningów.
Przebieg	1. Wybiera dodanie treningu. 2. Podaje nazwę, termin i ćwiczenia. 3. Określa serie, powtórzenia, obciążenie i odpoczynek. 4. Zapisuje plan.
Rezultat	Plan pojawia się na liście treningów oczekujących.
Ciężkie przypadki	Brak ćwiczeń, niepoprawna data, wartości ujemne, anulowanie formularza i konflikt z edytowanym planem.

PU-02: Wykonanie i zakończenie sesji

Element	Opis
Aktor	Użytkownik posiadający zaplanowany trening.
Warunek początkowy	Plan zawiera co najmniej jedno poprawne ćwiczenie.
Przebieg	1. Użytkownik uruchamia trening. 2. System pokazuje bieżący krok i timer. 3. Użytkownik zapisuje wynik serii. 4. System przechodzi przez odpoczynek i kolejne kroki. 5. Użytkownik zatwierdza zakończenie.
Rezultat	Sesja trafia do historii, a statystyki i seria regularności są aktualizowane.
Ciężkie przypadki	Timer schodzi poniżej zera, aplikacja zostaje zamknięta, zapis jest chwilowo w toku albo użytkownik wraca do już aktywnej sesji.

4.1. Przypadki użycia - analiza i postęp

PU-03: Kontynuowanie przerwanej sesji

Element	Opis
Aktor	Użytkownik z aktywną, niezakończoną sesją.
Przebieg	1. Użytkownik opuszcza widok sesji. 2. System zachowuje bieżący krok, fazę, pozostały czas, wpisy i notatki. 3. Pływający licznik wskazuje aktywność. 4. Po powrocie stan jest odtwarzany z uwzględnieniem czasu, który upłynął.
Rezultat	Użytkownik kontynuuje ten sam trening bez utraty wykonanych serii.

PU-04: Analiza nagrania ćwiczenia

Element	Opis
Aktor	Uwierzytelniony użytkownik podczas lub po treningu.
Przebieg	1. Wybiera typ ćwiczenia i nagranie. 2. API sprawdza format i rozmiar. 3. Nagranie zostaje zapisane. 4. Backend uruchamia analizator Python. 5. Wynik i metryki są zapisane w bazie.
Rezultat	Użytkownik otrzymuje status, podsumowanie, metryki i opcjonalne nagranie z oznaczeniami.
Ciężkie przypadki	Nieobsługiwany format, zbyt duży plik, brak modelu, przekroczenie czasu, błąd procesu lub brak pliku wyniku.

PU-05: Przegląd postępu

Element	Opis
Aktor	Użytkownik posiadający ukończone treningi.
Przebieg	Wybiera zakres danych, przegląda liczbę treningów, objętość, rekordy i serię regularności.
Rezultat	Statystyki są obliczone wyłącznie z treningów użytkownika i zgodnie z wybranym zakresem.

5. Architektura rozwiązania

System ma architekturę warstwową. Frontend odpowiada za prezentację, lokalny stan interakcji i wywołania API. Backend realizuje reguły dostępu, trwałość danych i agregacje. Moduł Python wykonuje kosztowną analizę obrazu i przekazuje wynik przez plik JSON.

Warstwa	Technologia	Odpowiedzialność
Interfejs	Next.js, React, TypeScript	Widoki, view modele, formularze, responsywność i cache zapytań.
API	ASP.NET Core 8	Autoryzacja, logika aplikacyjna, walidacja, kontrakty DTO i obsługa błędów.
Dane	Entity Framework Core, PostgreSQL	Trwałość kont, planów, sesji, wpisów i analiz.
Analiza	Python, OpenCV, YOLO	Przetwarzanie nagrania, metryki ruchu i przygotowanie materiału wynikowego.
Mobile	Capacitor	Uruchomienie wspólnego frontendu w aplikacji Android.

Przepływ aktywnej sesji

Frontend -> API -> baza

View model zapisuje fazę, indeks kroku, pozostały czas, stan pauzy, notatki i wpisy. API wiąże rekord z właścicielem i utrwala go. Po ponownym wejściu frontend pobiera aktywną sesję i rekonstruuje timer.

Przepływ analizy wideo

Nagranie -> API -> proces Python -> wynik

API waliduje plik i zapisuje rekord ze statusem przetwarzania. Proces Python otrzymuje ścieżki jako osobne argumenty, zapisuje wynik JSON i opcjonalne nagranie. Backend interpretuje rezultat, aktualizuje bazę i zwraca bezpieczny kontrakt DTO.

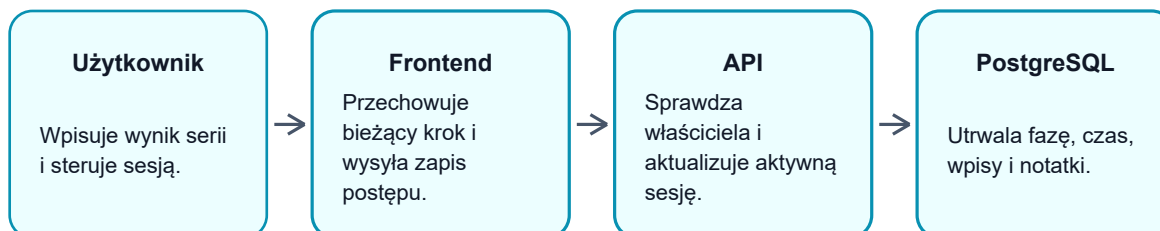
Granice odpowiedzialności

Interfejs nie oblicza statystyk globalnych i nie decyduje o prawach dostępu. Backend nie steruje animacjami ani rozmieszczeniem elementów mobilnych. Python nie komunikuje się bezpośrednio z bazą danych. Takie rozdzielenie ogranicza sprzężenie i ułatwia testowanie poszczególnych modułów.

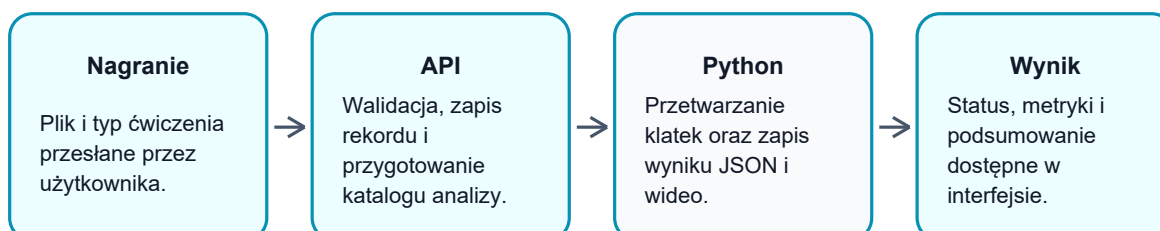
5.1. Przepływ danych

Poniższe schematy pokazują trzy najważniejsze przepływy. Nie przedstawiają wszystkich klas i endpointów, lecz miejsca, w których dane zmieniają właściciela albo sposób przechowywania.

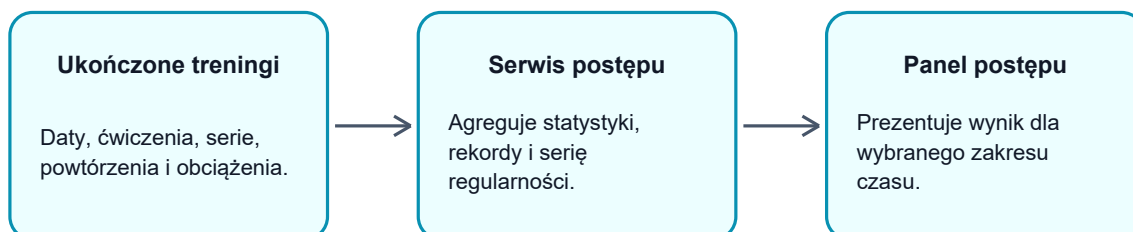
Aktywna sesja treningowa



Analiza nagrania



Postęp użytkownika



Każdy przepływ jest związany z identyfikatorem użytkownika. Frontend nie przekazuje właściciela jako dowolnej wartości; backend ustala go na podstawie uwierzytelnionego żądania.

6. Trudniejsze elementy implementacji

6.1. Odtwarzanie aktywnej sesji i czasu

Najbardziej wymagającym elementem interfejsu było zachowanie spójnego czasu po opuszczeniu strony. Sam stan komponentu React nie wystarcza, ponieważ znika przy przeładowaniu. W bazie przechowywany jest więc ostatni znany czas i znacznik aktualizacji. Podczas odtwarzania odejmowany jest czas, który upłynął od ostatniego zapisu.

Plik: frontend-app/src/app/components/pagesComponents/WorkoutRunPage/WorkoutRunPageVM.ts

```
const getRestoredRemainingSeconds = (
  run: WorkoutRunStart,
  phase: WorkoutRunPhase,
  fallbackSeconds: number,
): number => {
  const storedSeconds =
    run.remainingSeconds ?? (phase === "summary" ? 0 : fallbackSeconds);

  if (phase === "summary" || run.isPaused || !run.lastProgressAt) {
    return storedSeconds;
  }

  const elapsedSinceLastProgress = Math.floor(
    Math.max(0, Date.now() - run.lastProgressAt.getTime()) / 1000,
  );

  return storedSeconds - elapsedSinceLastProgress;
};
```

Funkcja rozróżnia sesję wstrzymaną, podsumowanie i aktywnie odmierzany etap. Wartość może stać się ujemna, ponieważ przekroczenie planowanego czasu nie powinno automatycznie zmieniać kroku bez decyzji użytkownika.

Dlaczego było to trudne?

Stan timera istnieje jednocześnie w pamięci komponentu, referencjach używanych przez autosave, cache zapytań i bazie danych. Każda z tych reprezentacji musi wskazywać ten sam krok i fazę także podczas asynchronicznego zapisu.

6.2. Trudniejsze elementy implementacji - postęp

Obliczanie bieżącej i najdłuższej serii regularności wymaga rozdzielenia dwóch pojęć. Bieżąca seria istnieje tylko wtedy, gdy ostatni trening wykonano dziś lub wczoraj. Najdłuższa seria jest historycznym maksimum i nie znika po przerwie.

Plik: backend-app/apiModule/ApiModule/WebApplication1/Application/ProgressService.cs

```
var days = workouts
    .Select(w => w.CompletedAt!.Value.Date)
    .Distinct()
    .OrderByDescending(d => d)
    .ToList();

var today = DateTime.UtcNow.Date;
var latestDay = days[0];
var current = 0;

if (latestDay == today || latestDay == today.AddDays(-1))
{
    current = 1;
    for (var i = 1; i < days.Count; i++)
    {
        if (days[i] != days[i - 1].AddDays(-1)) break;
        current++;
    }
}

var longest = 1;
var run = 1;
for (var i = 1; i < days.Count; i++)
{
    if (days[i] == days[i - 1].AddDays(-1))
    {
        run++;
        longest = Math.Max(longest, run);
    }
    else run = 1;
}
```

Daty są najpierw normalizowane i deduplikowane, dlatego kilka treningów jednego dnia zwiększa statystyki, ale nie wydłuża serii o kilka dni. Obliczenie odbywa się w UTC, zgodnie ze sposobem zapisu dat po stronie API.

Ciężkie przypadki

Brak ukończonych treningów, kilka treningów tego samego dnia, ostatni trening wczoraj, przerwa dłuższa niż jeden dzień oraz długa seria historyczna niezależna od serii bieżącej.

6.3. Trudniejsze elementy implementacji - analiza wideo

Moduł analizy wideo działa jako proces zewnętrzny względem API. Problemem nie było wyłącznie uruchomienie skryptu, lecz kontrola czasu, bezpieczne przekazanie argumentów, odczyt obu strumieni wyjściowych i obsługa sytuacji, w której proces zakończył się bez oczekiwanego pliku JSON.

Plik: backend-app/apiModule/ApiModule/WebApplication1/Application/FormAnalysis/FormAnalysisService.cs

```
using var process = new Process
{
    StartInfo = new ProcessStartInfo
    {
        FileName = pythonExecutable,
        WorkingDirectory = Path.GetDirectoryName(scriptPath) ?? analysisDir,
        RedirectStandardError = true,
        RedirectStandardOutput = true,
        UseShellExecute = false,
        CreateNoWindow = true,
    },
};

process.StartInfo.ArgumentList.Add(scriptPath);
process.StartInfo.ArgumentList.Add("--input");
process.StartInfo.ArgumentList.Add(sourcePath);
process.StartInfo.ArgumentList.Add("--output-dir");
process.StartInfo.ArgumentList.Add(analysisDir);

process.Start();
var stdoutTask = process.StandardOutput.ReadToEndAsync(ct);
var stderrTask = process.StandardError.ReadToEndAsync(ct);
var completed = await Task.Run(
    () => process.WaitForExit(timeoutSeconds * 1000), ct);

if (!completed || process.ExitCode != 0 || !File.Exists(resultJsonPath))
    return CreateScriptFailedResult(analysisId, exerciseType);
```

ArgumentList ogranicza ryzyko błędnego cytowania ścieżek i wstrzyknięcia poleceń. Odczyt stdout i stderr rozpoczyna się przed oczekiwaniem na proces, co zapobiega blokadzie pełnych buforów. Każde niepowodzenie jest zamieniane na kontrolowany wynik domenowy zamiast surowego wyjątku.

Kontrola zasobów

Przed uruchomieniem analizatora sprawdzany jest format i rozmiar nagrania. Konfiguracja ogranicza czas procesu, liczbę analiz użytkownika i czas przechowywania plików.

6.4. Trudniejsze elementy implementacji - interfejs

Pływający licznik musi pozostać widoczny po zmianie rozmiaru okna, obrocie telefonu i przy dolnej nawigacji mobilnej. Pozycja zapisana w localStorage może być poprawna dla poprzedniego ekranu, ale znajdować się poza aktualnym obszarem roboczym. Dlatego każda pozycja jest ograniczana do bieżących wymiarów.

Plik: frontend-app/src/app/components/ActiveWorkoutFloatingTimer/ActiveWorkoutFloatingTimer.tsx

```
const clampPosition = (position: FloatingPosition): FloatingPosition => {
  if (typeof window === "undefined") return position;

  return {
    x: Math.min(
      Math.max(EDGE_GAP, position.x),
      Math.max(EDGE_GAP,
        window.innerWidth - getFloatingWidth() - EDGE_GAP),
    ),
    y: Math.min(
      Math.max(EDGE_GAP, position.y),
      Math.max(EDGE_GAP,
        window.innerHeight - HEIGHT - getBottomReservedSpace()),
    ),
  };
};

window.addEventListener("resize", handleResize);
return () => window.removeEventListener("resize", handleResize);
```

Dolna rezerwa jest większa na telefonie, aby licznik nie zasłaniał nawigacji. Po zmianie wymiarów pozycja jest obliczana ponownie, a podczas przeciągania aktualizacje są grupowane przez requestAnimationFrame, co ogranicza liczbę renderów.

Dostępność

Element obsługuje kliknięcie, klawisze Enter i Spacja oraz posiada osobny przycisk zamknięcia. Przeciągnięcie nie jest interpretowane jako przejście do aktywnego treningu.

7. Weryfikacja i ciężkie przypadki

Testy jednostkowe obejmują logikę view modeli, serwisów trybu demonstracyjnego i warstwy backendowej. Dodatkowo istotne scenariusze są sprawdzane w pełnym przepływie z API, bazą danych i interfejsem. Zakres testów jest dobierany do ryzyka utraty danych lub przerwania aktywnej sesji.

Obszar	Sprawdzane zachowanie
Sesja	Odtworzenie fazy, kroku i czasu; brak duplikacji wpisów; zapis notatek; zakończenie tylko po potwierdzeniu.
Timer	Wartości dodatnie i ujemne, pauza, zmiana zakładki, odświeżenie oraz zmiana rozmiaru ekranu.
Postęp	Brak danych, duplikaty dat, trening dziś/wczoraj, przerwana seria i najdłuższa seria historyczna.
Dostęp	Brak możliwości odczytu cudzego treningu, sesji, profilu lub analizy.
Formularze	Puste pola, liczby ujemne, niepoprawny termin i anulowanie bez częściowego zapisu.
Analiza wideo	Brak pliku, zły format, przekroczony rozmiar, brak skryptu, timeout, kod wyjścia różny od zera i brak JSON.
Mobile	Dolna nawigacja, safe area, długie etykiety, obrót urządzenia i brak nachodzenia elementów.

Kryteria zakończenia

Scenariusz uznaje się za poprawny, gdy użytkownik otrzymuje jednoznaczny rezultat, dane pozostają spójne po ponownym odczycie, a błąd zewnętrznej usługi nie prowadzi do utraty pozostałych informacji. W przypadku sesji treningowej dodatkowym kryterium jest możliwość bezpiecznej kontynuacji po opuszczeniu widoku.

Ograniczenie analizy wideo

Analiza ma charakter wspomagający i eksperymentalny. Nie zastępuje oceny trenera ani specjalisty medycznego; jakość wyniku zależy od kadru, oświetlenia, widoczności sylwetki i obsługiwanego typu ćwiczenia.

CZĘŚĆ II

Instrukcja użytkownika i uruchomienia

Ta część przedstawia ekrany aplikacji, najważniejsze działania użytkownika, wariant mobilny oraz sposób uruchomienia środowiska demonstracyjnego i trybu z API.

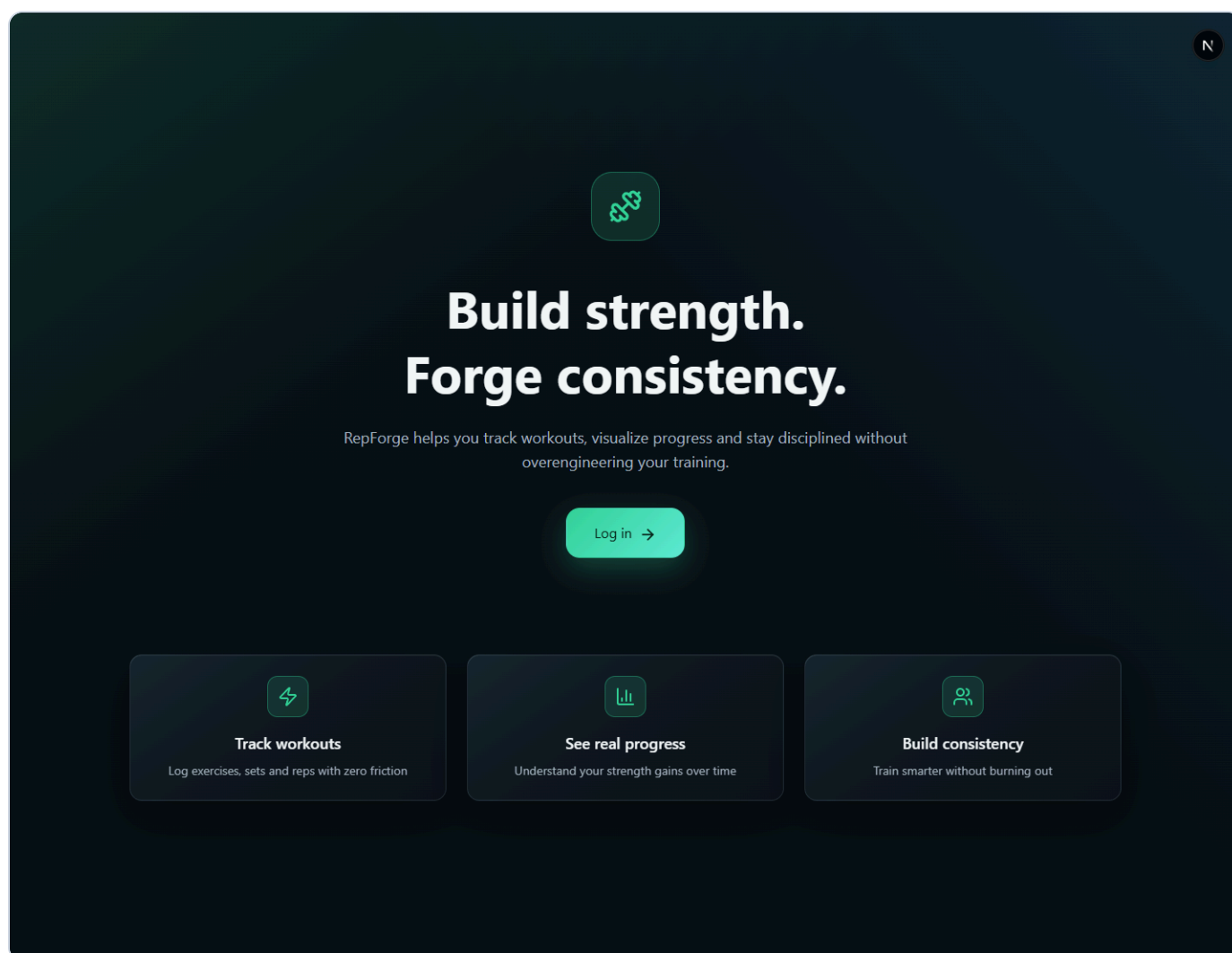
Numeracja rozdziałów w tej części rozpoczyna się ponownie od 1, ponieważ jest to samodzielna instrukcja korzystania z aplikacji.

1. O czym jest RepForge

RepForge powstał jako aplikacja do ogarniania treningu siłowego od początku do końca. Nie chodzi tylko o listę ćwiczeń. Użytkownik może zaplanować trening, odpalić go w trybie sesji, zapisać realnie wykonane serie, zrobić notatki, a później sprawdzić postęp bez przepisywania danych do osobnego arkusza.

Projekt jest przygotowany w trzech wariantach użycia: klasyczny web na komputerze, web mobile w przeglądarce telefonu oraz aplikacja mobilna przez Capacitor. To nadal jeden frontend, ale dopasowany wizualnie i technicznie do różnych miejsc uruchomienia.

Pierwszy ekran pełni rolę wejścia do aplikacji: pokazuje nazwę produktu, krótko ustawia kontekst i prowadzi użytkownika do logowania albo rejestracji.



Ekran startowy. Pierwszy widok aplikacji RepForge z komunikatem produktowym i wejściem do procesu logowania.

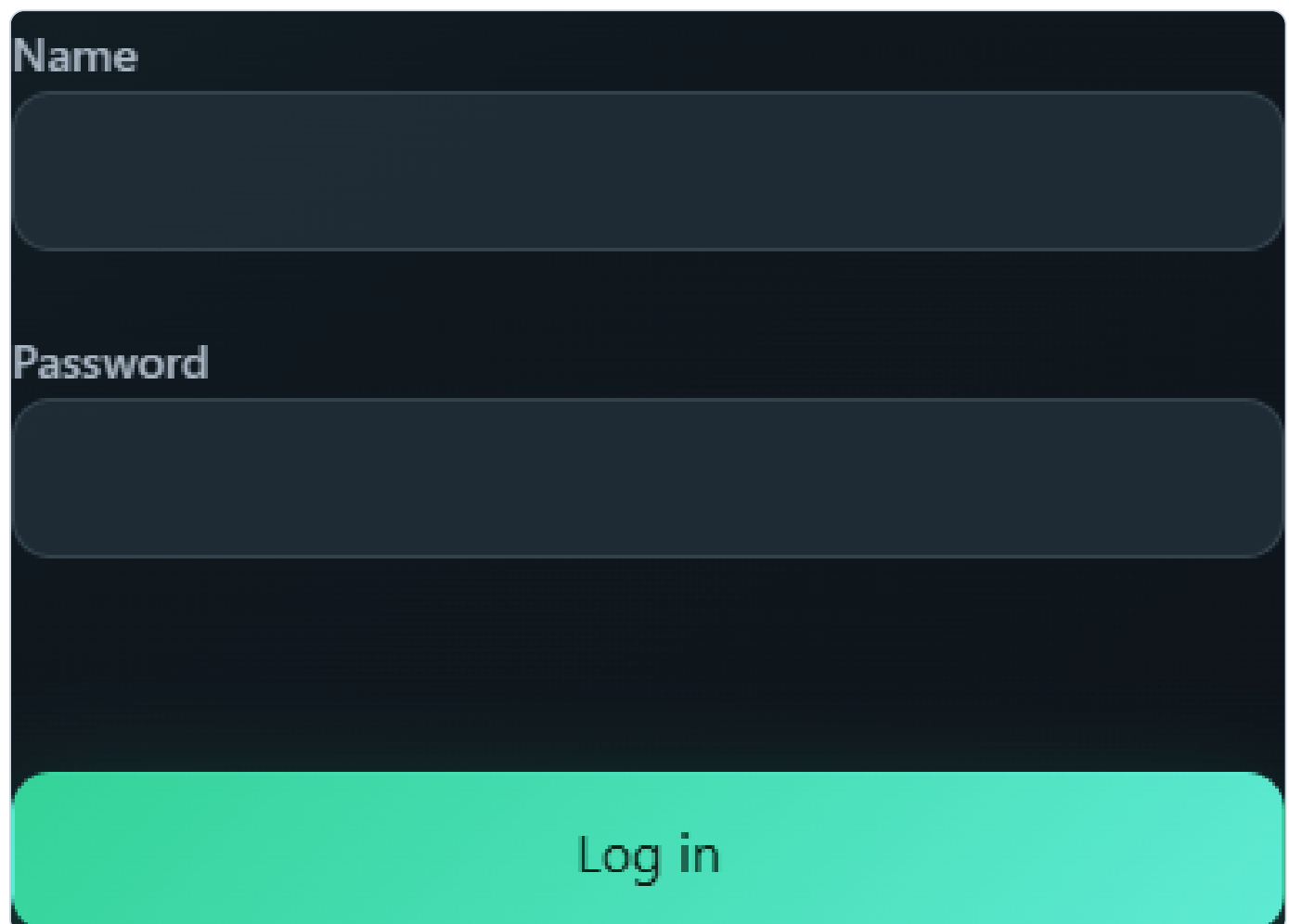
2. Logowanie i główny przepływ

Aplikacja ma jedną główną rolę: zalogowany użytkownik prowadzący własny plan treningowy. W trybie mock działa użytkownik demonstracyjny, a w trybie real konto jest obsługiwane przez backend ASP.NET Core i token JWT.

Podstawowy scenariusz wygląda tak:

1. użytkownik loguje się albo zakłada konto,
2. na dashboardzie widzi, czy ma dzisiaj trening, odpoczynek albo aktywną sesję,
3. w Workouts tworzy, poprawia lub uruchamia trening,
4. w trybie sesji zapisuje serie i notatki,
5. po treningu sprawdza statystyki oraz rekordy w Progress,
6. w Profile może uzupełnić dane konta i zmienić hasło.

Formularz logowania jest celowo prosty. W błędnych przypadkach użytkownik dostaje opis problemu, a nie surowy błąd API.



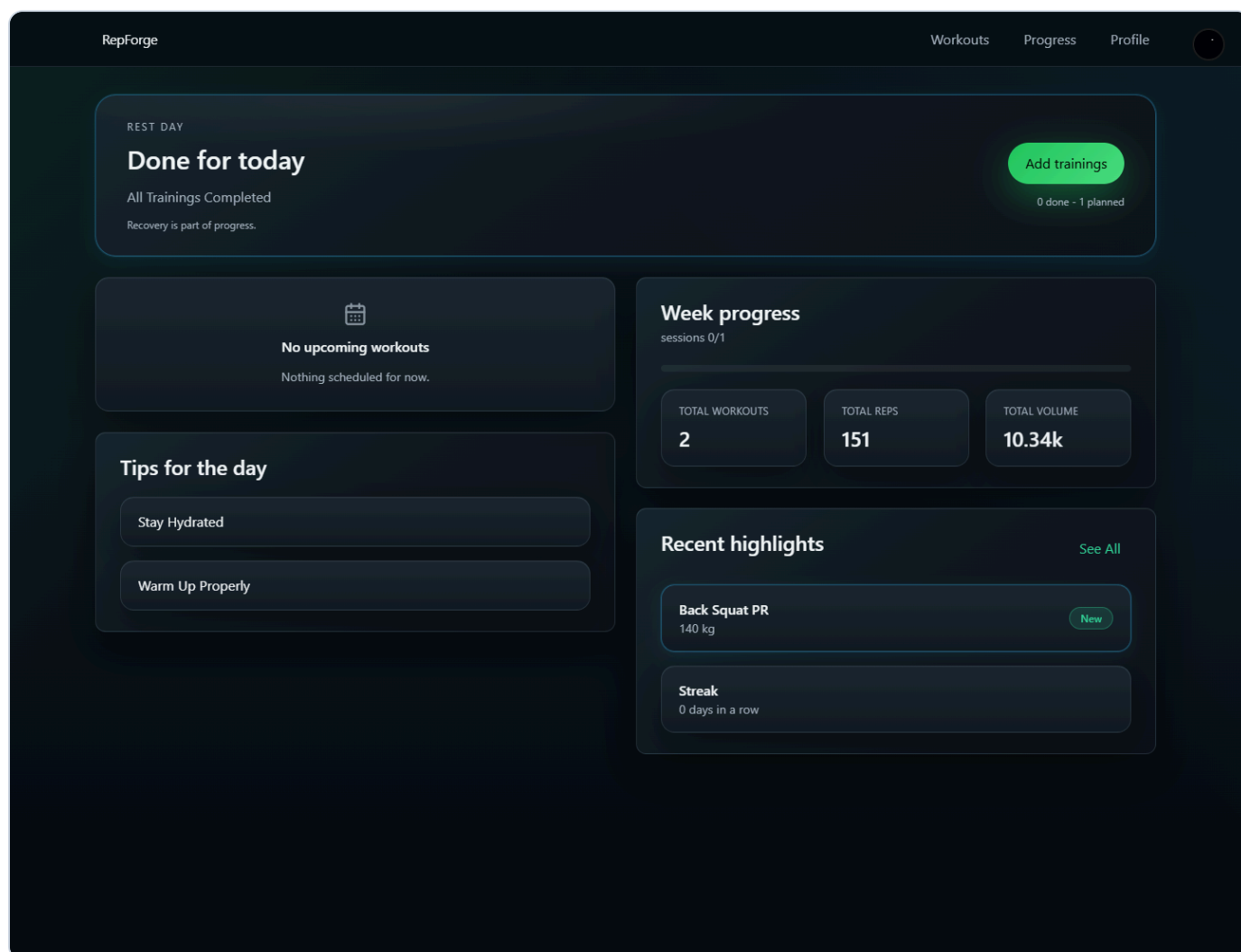
Logowanie i rejestracja. Widok uwierzytelniania z przełącznikiem logowania i rejestracji oraz walidowanymi polami formularza.

3. Dashboard

Dashboard jest miejscem, w którym użytkownik po zalogowaniu powinien od razu wiedzieć, co zrobić dalej. Jeśli nie ma nic zaplanowanego, aplikacja proponuje dodanie treningu. Jeśli trening trwa, dashboard nie udaje, że dzień jest zakończony, tylko pokazuje aktywną sesję i przycisk powrotu do niej.

To jest ważny detal UX, bo trening może zostać przerwany przez odświeżenie strony, przejście do innej zakładki albo wyjście do listy treningów. Aplikacja przechowuje stan sesji i potrafi wrócić do właściwego miejsca zamiast zaczynać trening od nowa.

Pełny dashboard pokazuje ogólną kompozycję: górny panel decyzyjny, dane tygodnia i skróty do dalszych działań.



Dashboard użytkownika. Panel główny agreguje najbliższy trening, tygodniowy postęp, wskazówki dnia i ostatnie osiągnięcia.

Tygodniowy panel jest osobnym, zwartym podsumowaniem. Nie trzeba wchodzić w historię, żeby zobaczyć, czy tydzień idzie zgodnie z planem.

Week progress

sessions 0/1

TOTAL WORKOUTS

2

TOTAL REPS

151

TOTAL VOLUME

10.34k

Tygodniowy postęp. Kadr samego panelu tygodnia pokazuje, jak aplikacja streszcza wykonane i zaplanowane jednostki bez wchodzenia w szczegóły treningu.

Ostatnie osiągnięcia są drobnym motywatorem: użytkownik widzi, że zapisane treningi faktycznie przekładają się na konkretne wyniki.

Recent highlights

[See All](#)

Back Squat PR

140 kg

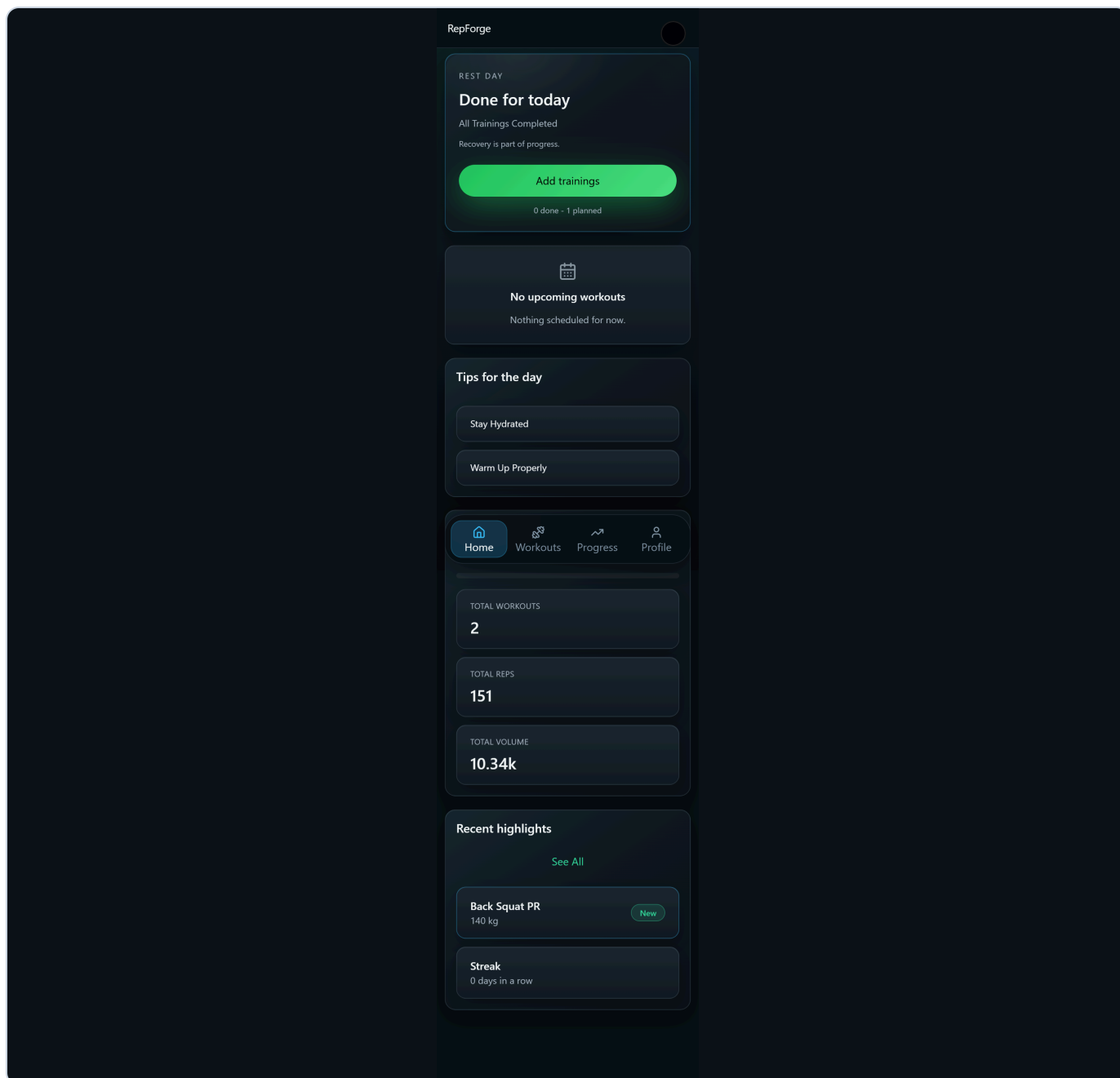
New

Streak

0 days in a row

Ostatnie osiągnięcia. Recent highlights są szybkim podglądem efektów pracy, czyli miejscem na rekordy i najnowsze pozytywne sygnały z historii.

Na telefonie dashboard przechodzi w układ pionowy i korzysta z dolnej nawigacji. To zachowuje charakter aplikacji mobilnej, a nie zwykłej strony zmniejszonej do szerokości telefonu.



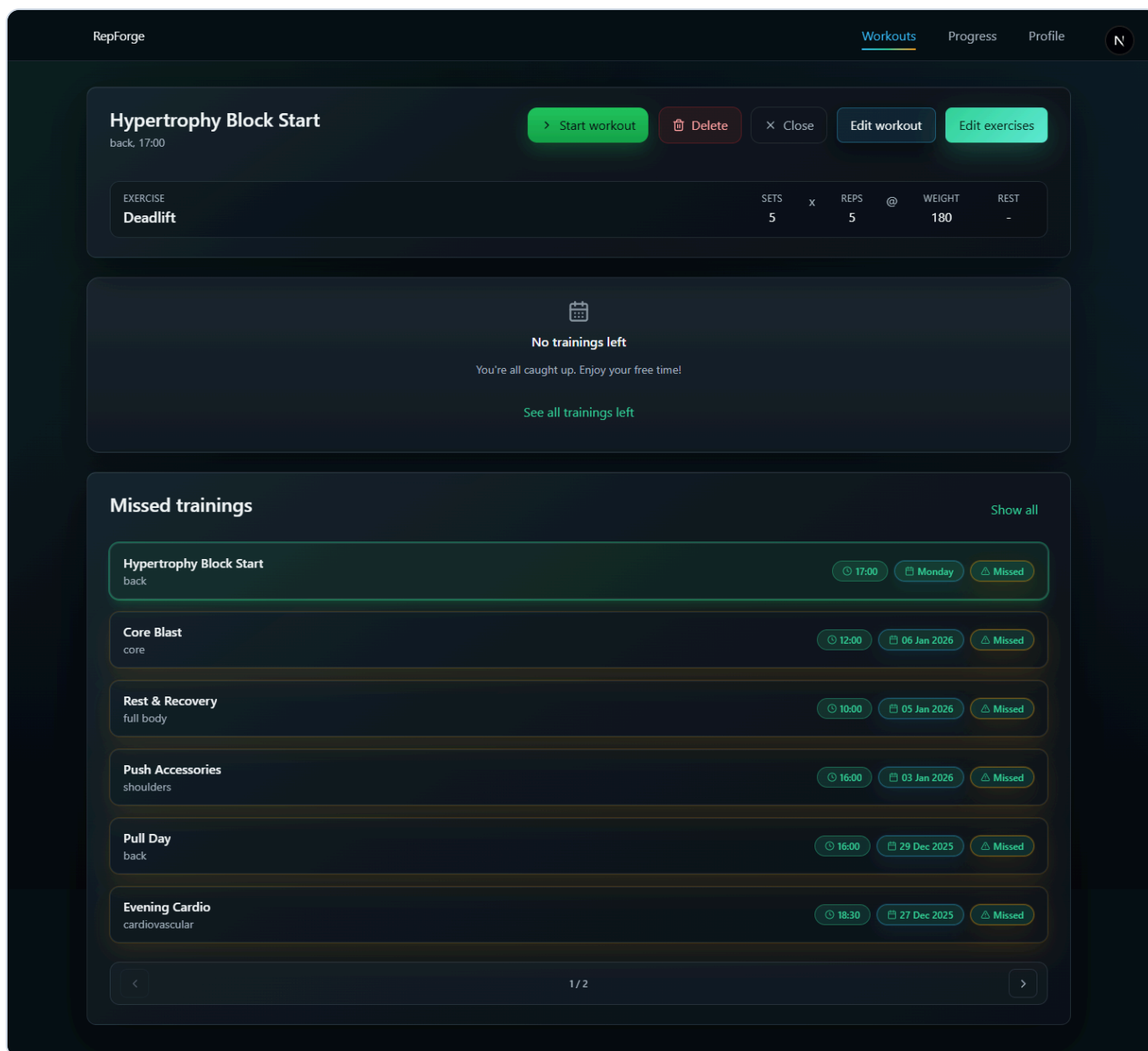
Dashboard na telefonie. Mobilny dashboard ma dolną nawigację, bezpieczne odstępy i układ dostosowany do obsługi kciukiem.

4. Treningi: plan, historia i edycja

Workouts to najważniejszy moduł roboczy. Tutaj użytkownik widzi treningi zaplanowane, pominięte i zakończone. Przy większej liczbie wpisów historia jest paginowana, żeby panel nie zamieniał się w długą ścianę kart.

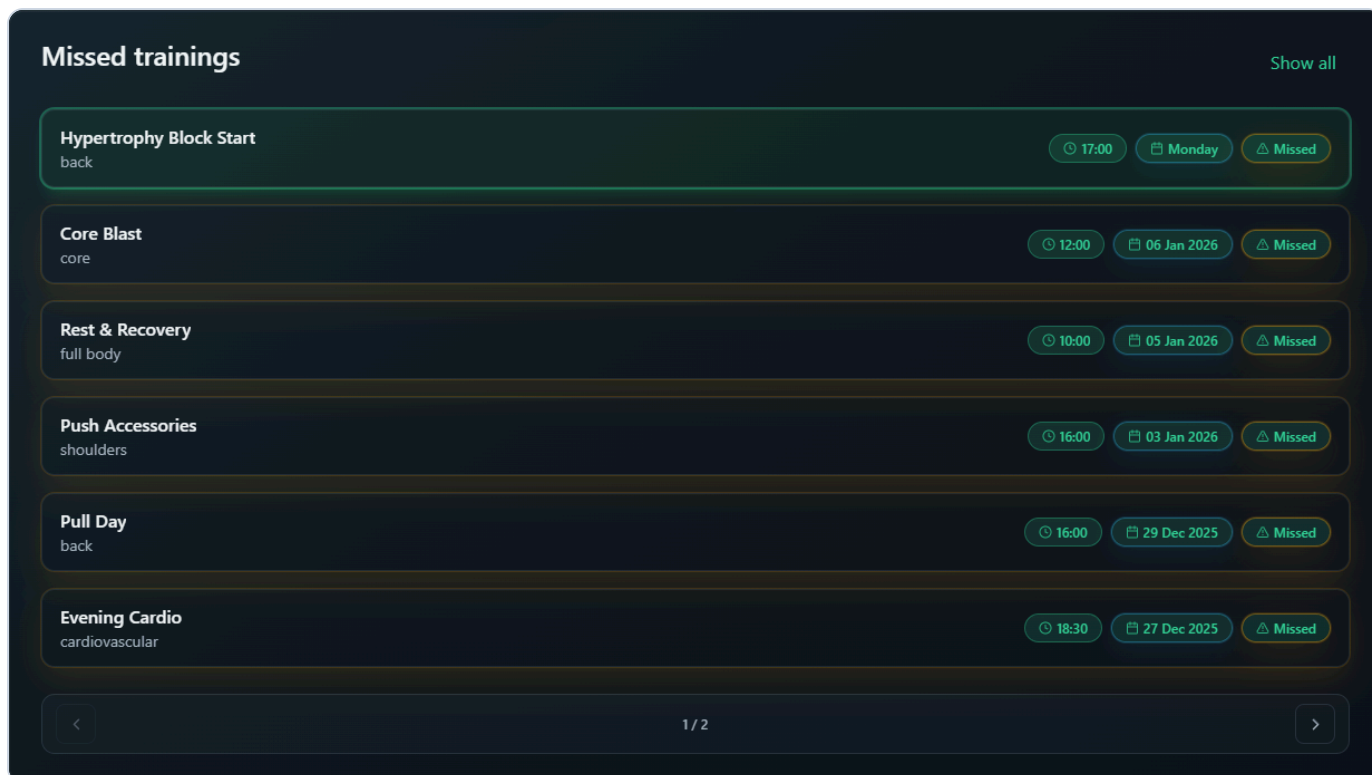
Widok szczegółów treningu ma być szybki w obsłudze: można wystartować lub kontynuować trening, usunąć go po potwierdzeniu, poprawić metadane albo wejść w edycję ćwiczeń. Na mobile te same akcje układają się pionowo, ponieważ na małym ekranie poziomy pasek przycisków byłby niewygodny i łatwo łamałby layout.

Widok przedstawia moduł Workouts wypełniony danymi demonstracyjnymi: aktywny wybór, panel treningu oraz listę historii.



Lista treningów. Ekran treningów pokazuje planowane, pominięte i ukończone jednostki oraz pozwala przejść do szczegółów.

Oddzielny kadr historii pokazuje, że pominięte treningi są widoczne, ale kontrolowane paginacją. To przydaje się przy bazie z dużą liczbą treningów testowych.



Historia i pominięte treningi. Ten panel jest ważny przy dużej bazie danych: użytkownik widzi, co zostało pominięte, ale lista nie rozpycha całej strony.

Tworzenie treningu działa w modalu, bo jest to czynność wykonywana z kontekstu listy. Użytkownik nie traci miejsca, z którego zaczął.

Create new training

Training name
Give your workout a clear, recognizable name
e.g. Push Day

Muscle groups
Select or type muscle group

Schedule
Choose when this training takes place
dd/mm/yyyy --:--

Exercises
Add one or more exercises

Exercise 1

Name

Sets* x Reps* @ Weight (kg) Rest (s)

Cancel Create training

Evening Cardio
cardiovascular 18:30 27 Dec 2025 Missed

1 / 2

Tworzenie treningu. Modal tworzenia treningu pozwala określić nazwę, datę, grupy mięśniowe, ćwiczenia, serie, powtórzenia i obciążenie.

Szczegóły treningu są punktem decyzyjnym: start, kontynuacja, edycja, usunięcie i lista ćwiczeń są w jednym miejscu.

Morning Strength
chest, 09:00

> Start workout Delete X Close Edit workout Edit exercises

EXERCISE	SETS	x	REPS	@	WEIGHT	REST
Bench Press	3		10		100	-
Dips	4		12		-	-

Szczegóły treningu. Widok szczegółów treningu zawiera akcje startu, edycji, usunięcia oraz listę ćwiczeń.

Edycja ćwiczeń jest rozdzielona od zwykłej edycji danych treningu, bo zmiana serii, powtórzeń i obciążeń ma większy wpływ na późniejszą sesję.

Morning Strength
chest, 09:00

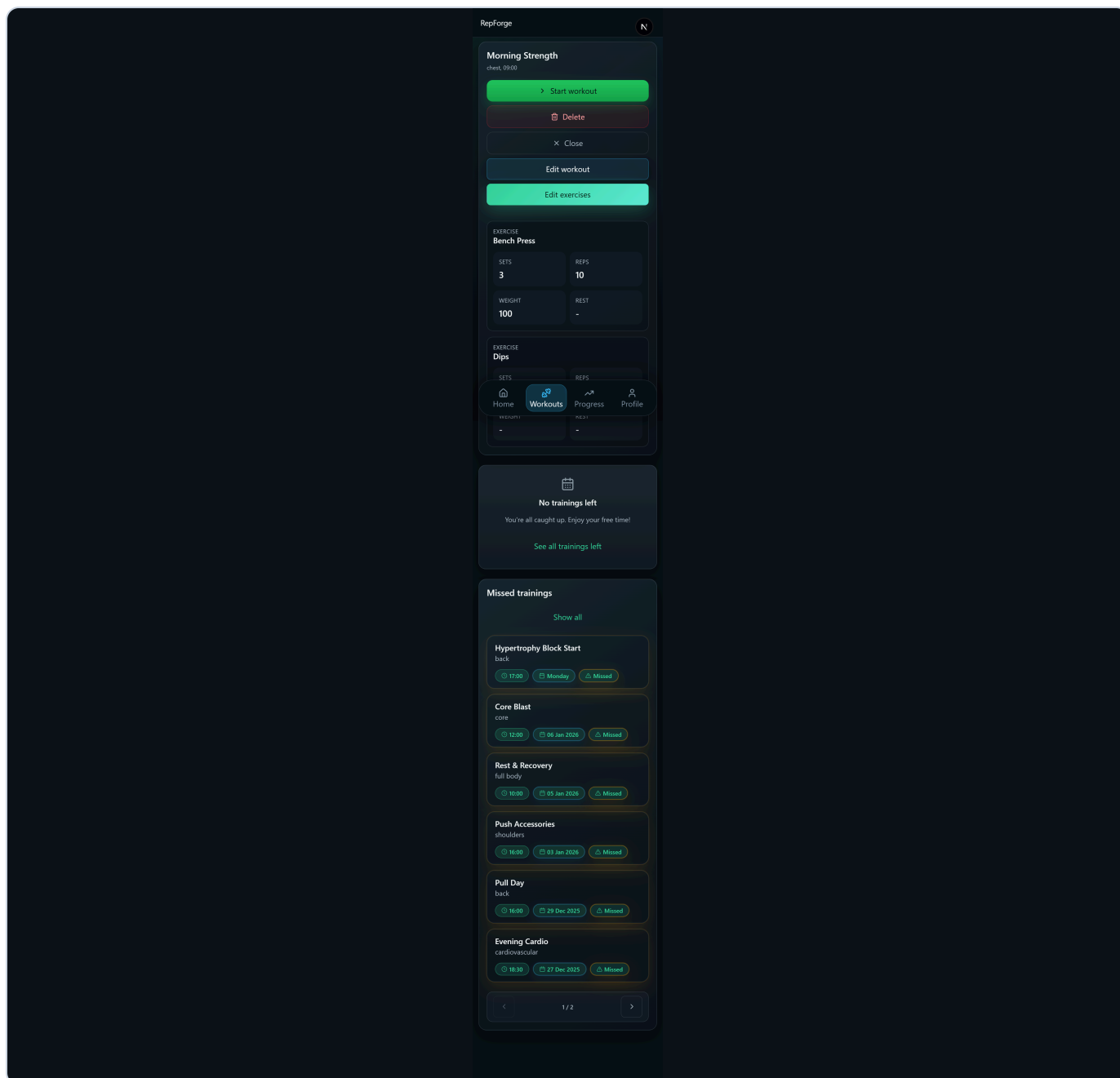
Edit workout Cancel

EXERCISE	SETS	x	REPS	@	WEIGHT	REST	x
Bench Press	3		10		100	0	
Dips	4		12		0	0	

+ Add exercise

Edycja ćwiczeń. Tryb edycji ćwiczeń umożliwia aktualizację struktury treningu z walidacją wymaganych pól.

Na telefonie ten sam widok jest bardziej pionowy. Chodzi o uniknięcie nakładania etykiet, wartości i ikon systemowych.



Szczegóły treningu na telefonie. Mobilny widok treningu układa akcje i ćwiczenia pionowo, bez nachodzenia etykiet na wartości.

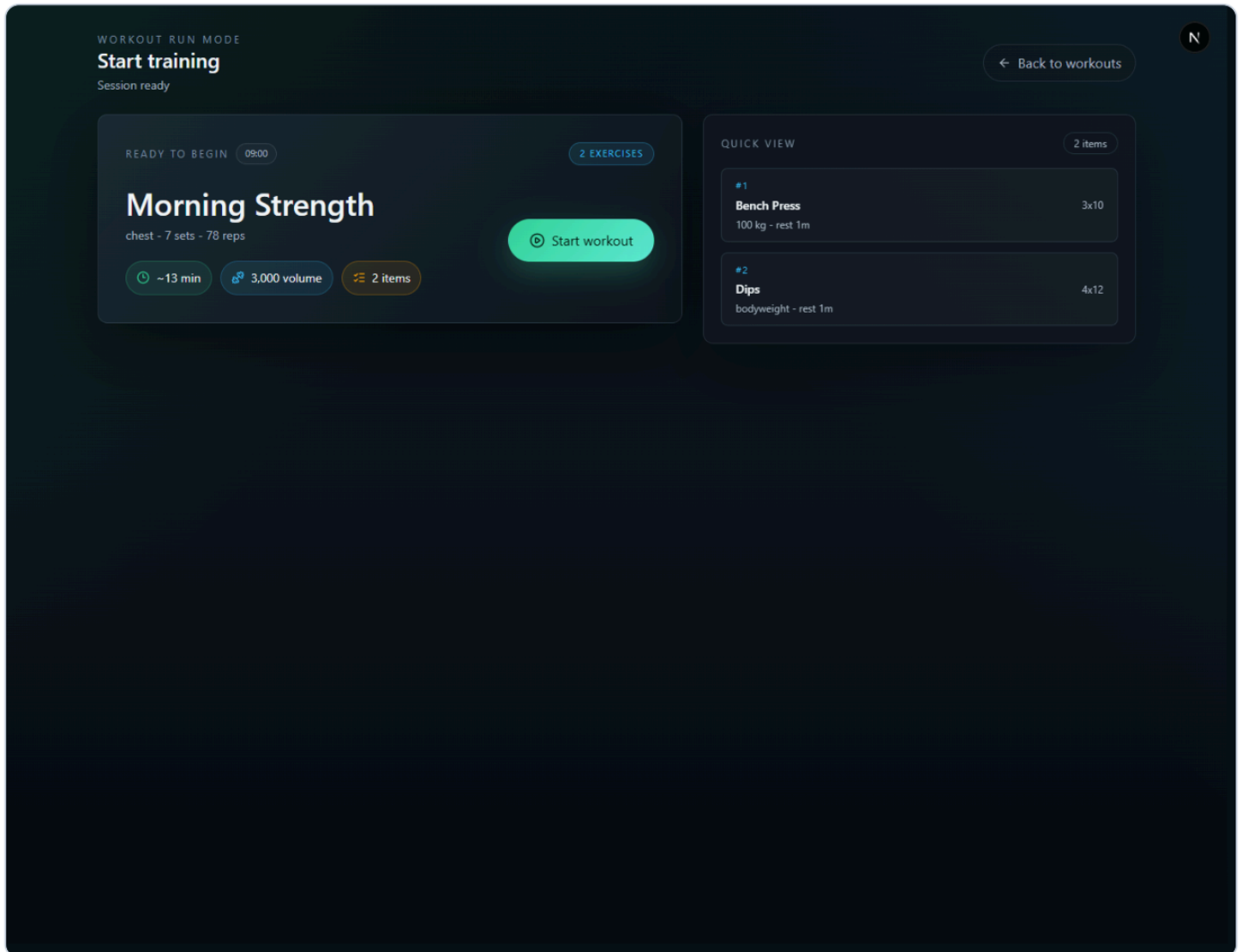
5. Sesja treningowa

Tryb sesji treningowej jest osobnym przepływem, bo użytkownik w trakcie ćwiczeń potrzebuje innego interfejsu niż przy planowaniu. Tu liczy się duży timer, aktualny cel, szybki zapis serii i mała liczba rzeczy do przemyślenia.

Timer serii jest estymowany na podstawie planu, ale nie zamyka etapu automatycznie. Jeżeli użytkownik potrzebuje więcej czasu, licznik może zejść poniżej zera. Dopiero świadoma akcja, np. Done albo Skip current set, przełącza sesję dalej. To lepiej pasuje do realnego treningu niż sztywne odliczanie.

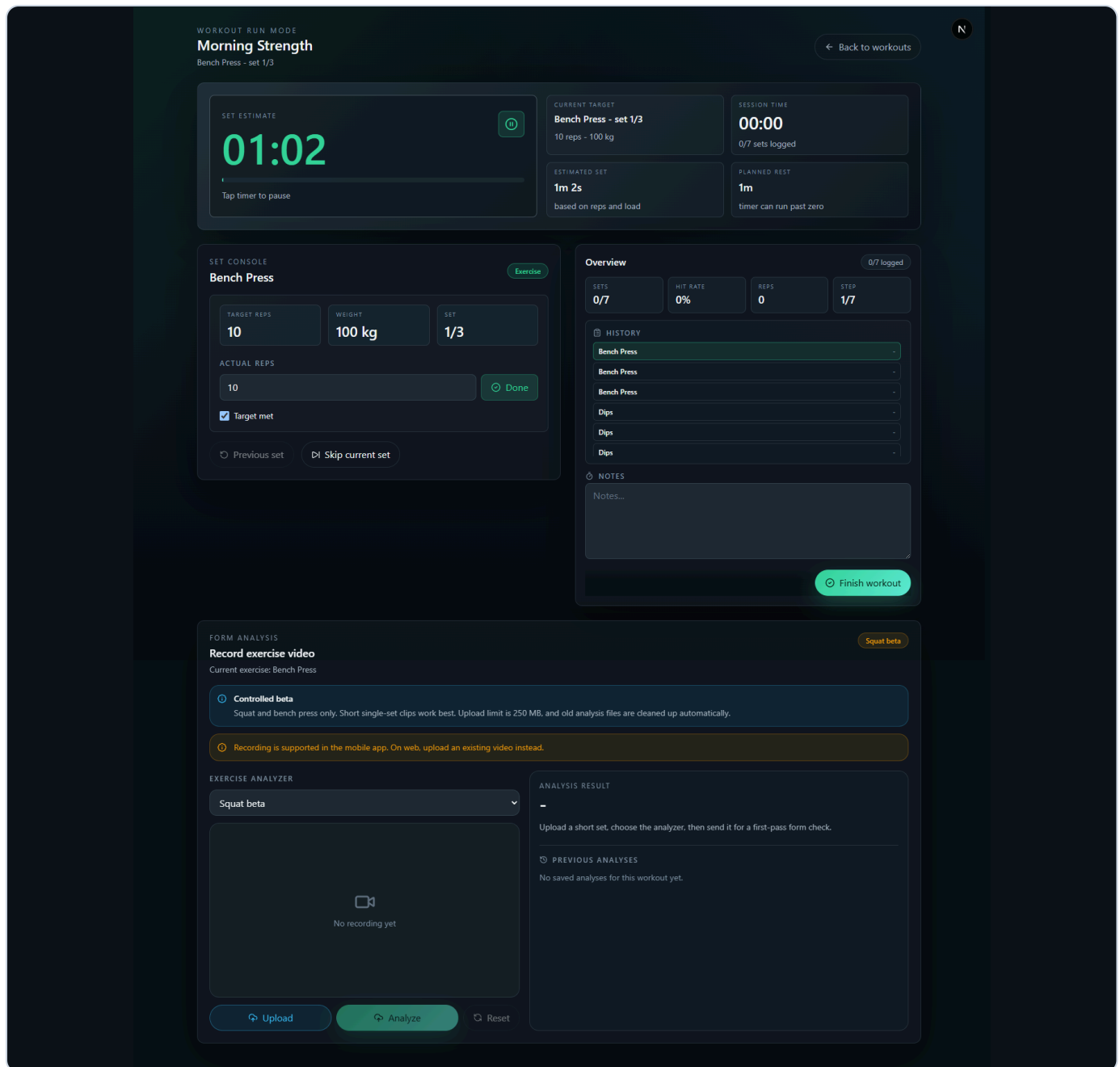
Ważny scenariusz: jeżeli użytkownik opuści widok treningu bez zakończenia sesji, aktywny trening dalej istnieje. Aplikacja może pokazać podręczny licznik czasu i pozwala wrócić do sesji bez utraty historii wykonanych serii.

Przed startem aplikacja pokazuje plan i nie udaje jeszcze, że sesja trwa. To bezpieczny moment na sprawdzenie, czy użytkownik wybrał właściwy trening.



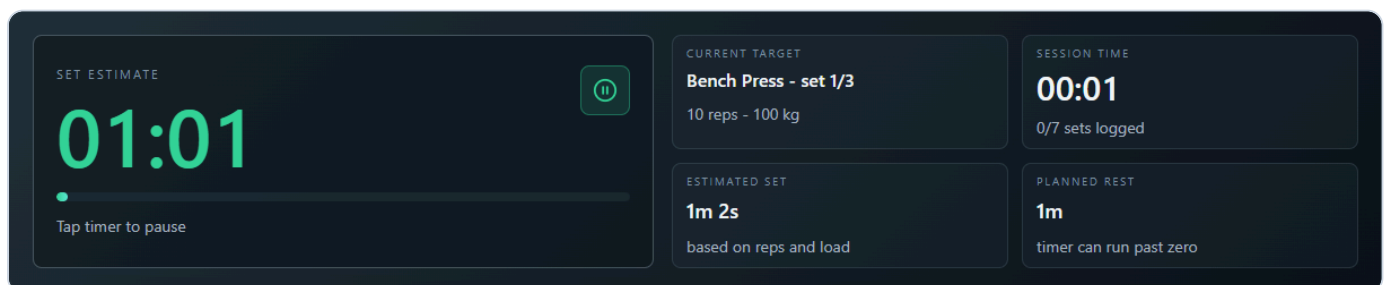
Przygotowanie sesji treningowej. Ekran startu sesji prezentuje podsumowanie planu i pozwala uruchomić trening.

Pełny widok aktywnej sesji pokazuje rozkład ekranu: timer na górze, konsola serii po lewej, historia i notatki po prawej.



Aktywna sesja treningowa. Aktywna sesja ma duży timer, bieżący cel, konsolę serii, historię wpisów i notatki.

Górny panel jest sercem sesji. W jednym miejscu widać czas, cel, liczbę zapisanych serii oraz planowaną przerwę.



Timer i cel serii. Górny panel sesji pokazuje timer, aktualny cel, czas całej sesji, estymację serii i zaplanowaną przerwę.

Konsola serii skupia się na jednym zadaniu: wpisać realną liczbę powtórzeń i zatwierdzić wykonanie.

SET CONSOLE

Exercise

Bench Press

TARGET REPS

10

WEIGHT

100 kg

SET

1/3

ACTUAL REPS

10

Done

☒ Target met

Previous set

Skip current set

Konsola wpisu serii. Konsola serii to miejsce, w którym użytkownik wpisuje realną liczbę powtórzeń i potwierdza wykonanie zestawu.

Historia i notatki są obok, ale nie konkurują z główną akcją. To miejsce na szybkie sprawdzenie, co już zostało zapisane.

Overview

0/7 logged

SETS

0/7

HIT RATE

0%

REPS

0

STEP

1/7

HISTORY

Bench Press

-

Bench Press

-

Bench Press

-

Dips

-

Dips

-

Dips

-

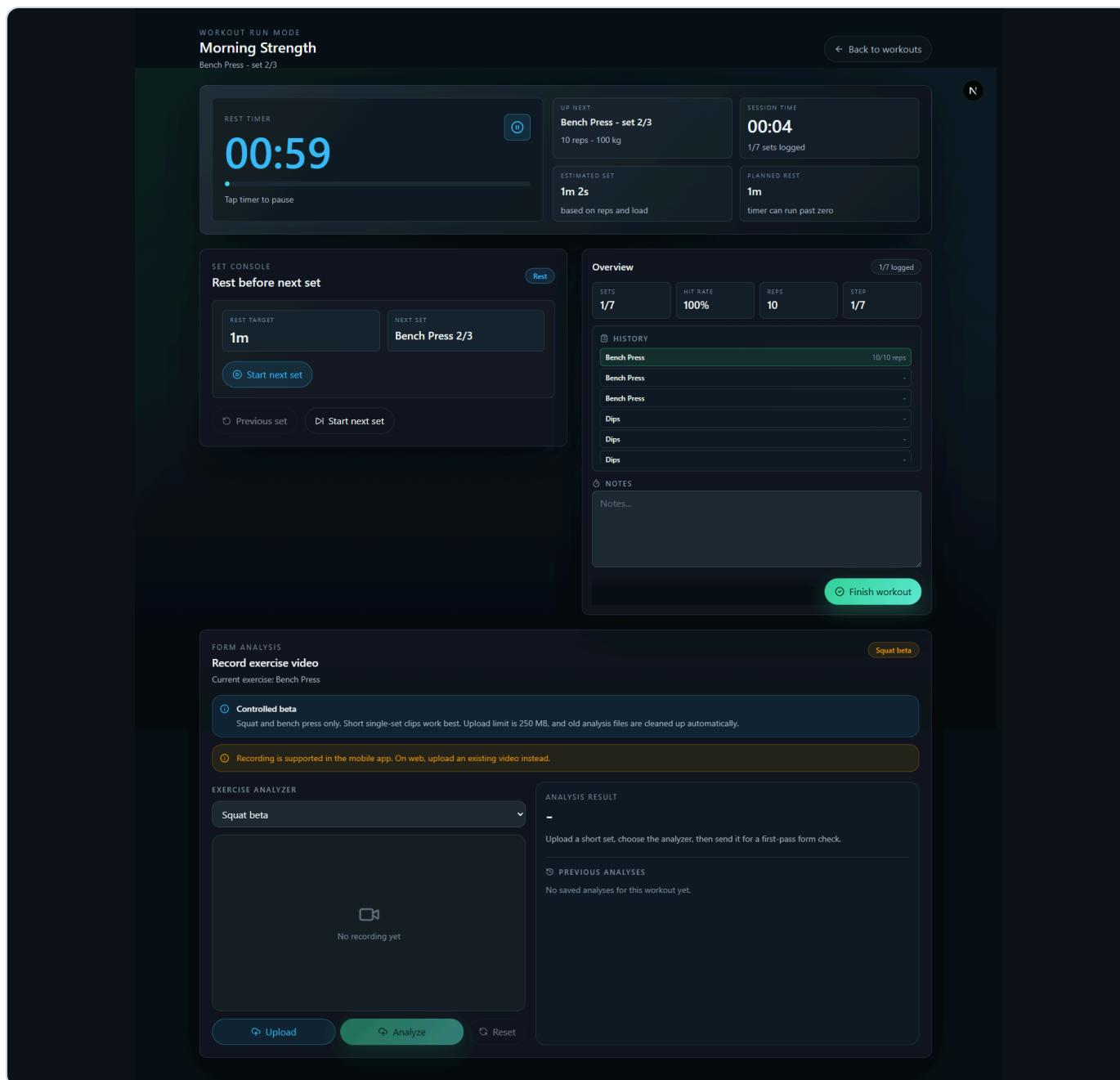
NOTES

Notes...

Finish workout

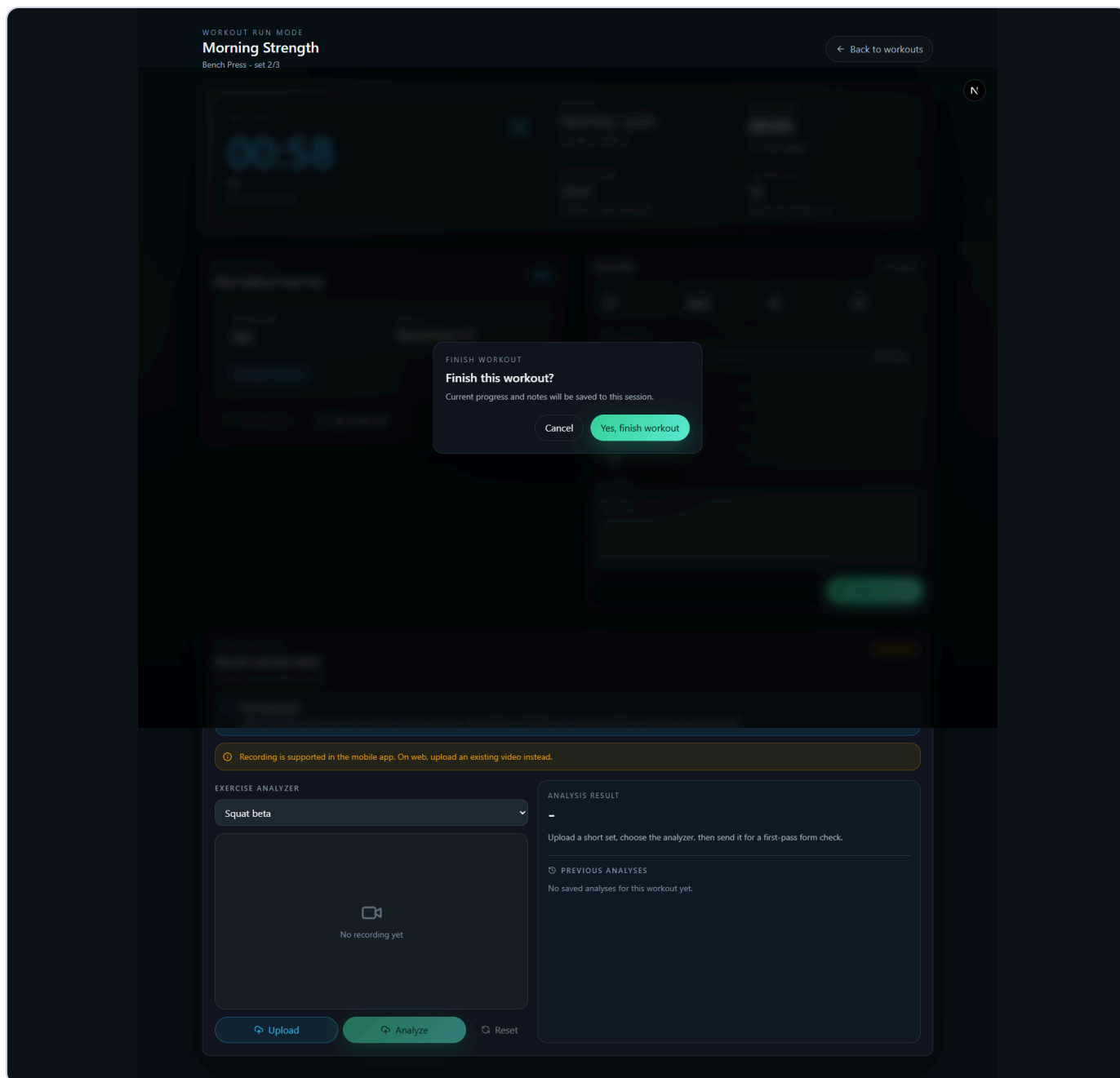
Historia i notatki sesji. Prawy panel zbiera podsumowanie, wpisy historii oraz notatki, żeby nie mieszać ich z szybką obsługą serii.

Po zatwierdzeniu serii historia od razu pokazuje wpis, a aplikacja przechodzi do kolejnego etapu sesji.



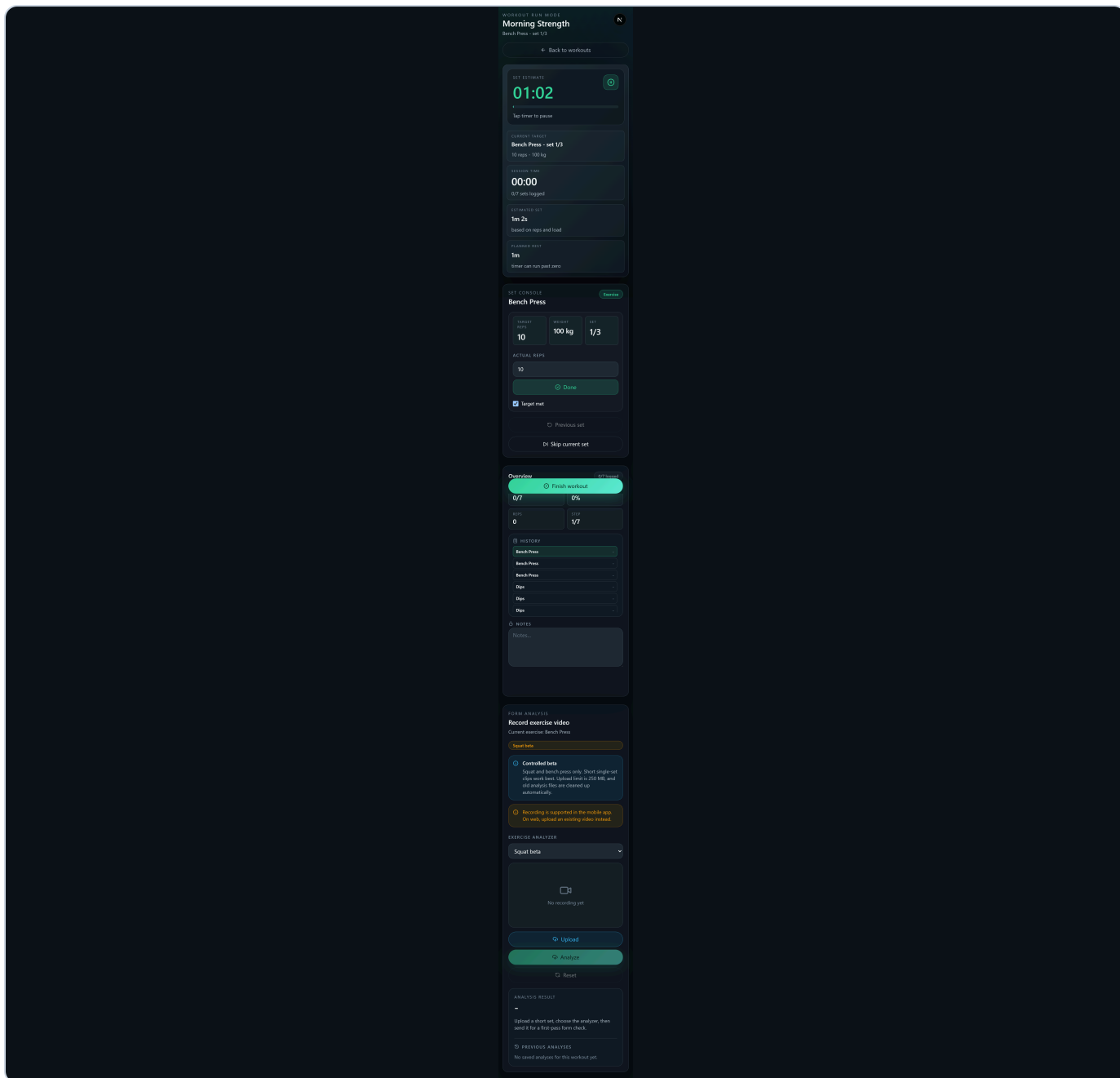
Zapis wykonanej serii. Po kliknięciu Done aplikacja zapisuje serię, aktualizuje historię i przechodzi do kolejnego etapu lub przerwy.

Zakończenie treningu ma osobne potwierdzenie, bo jest to akcja kończąca cały przebieg i zapisująca wynik.



Potwierdzenie zakończenia treningu. Zakończenie treningu jest zabezpieczone dialogiem potwierdzającym, żeby użytkownik nie zamknął sesji przypadkiem.

Na telefonie sesja zachowuje tę samą logikę, ale układ jest podporządkowany szybkiemu klikaniu jedną ręką.



Sesja treningowa na telefonie. Tryb sesji na telefonie eksponuje timer, aktualny cel i przyciski akcji w jednym przepływie.

6. Analiza formy ćwiczenia

Analiza wideo jest oznaczona jako beta, bo to najbardziej eksperymentalna część projektu. Użytkownik może nagrać lub przesłać film, wybrać typ ćwiczenia i uruchomić analizę. Aktualnie obsługiwane są podstawowe warianty: przysiad oraz wyciskanie leżąc.

Backend zapisuje metadane analizy, a moduł Python/OpenCV/YOLO próbuje ocenić ruch. Dla przysiadu sprawdzana jest między innymi głębokość i kontrola pozycji, a dla wyciskania leżąc prosta analiza łokci oraz pełnego zakresu ruchu. Nie jest to medyczny ani trenerski werdykt, tylko MVP pokazujące, jak można podpiąć analizę obrazu do aplikacji treningowej.

Ta funkcja ma ograniczenia: maksymalny rozmiar przesyłanego pliku, czas przechowywania starszych analiz i kontrolowane stany błędów. Jeśli skrypt nie zadziała, użytkownik widzi informację o niepowodzeniu analizy beta, a nie techniczny ślad błędu.

Panel analizy jest w sesji treningowej, ale nie blokuje wykonania treningu. To dodatkowe narzędzie, nie warunek zapisu serii.

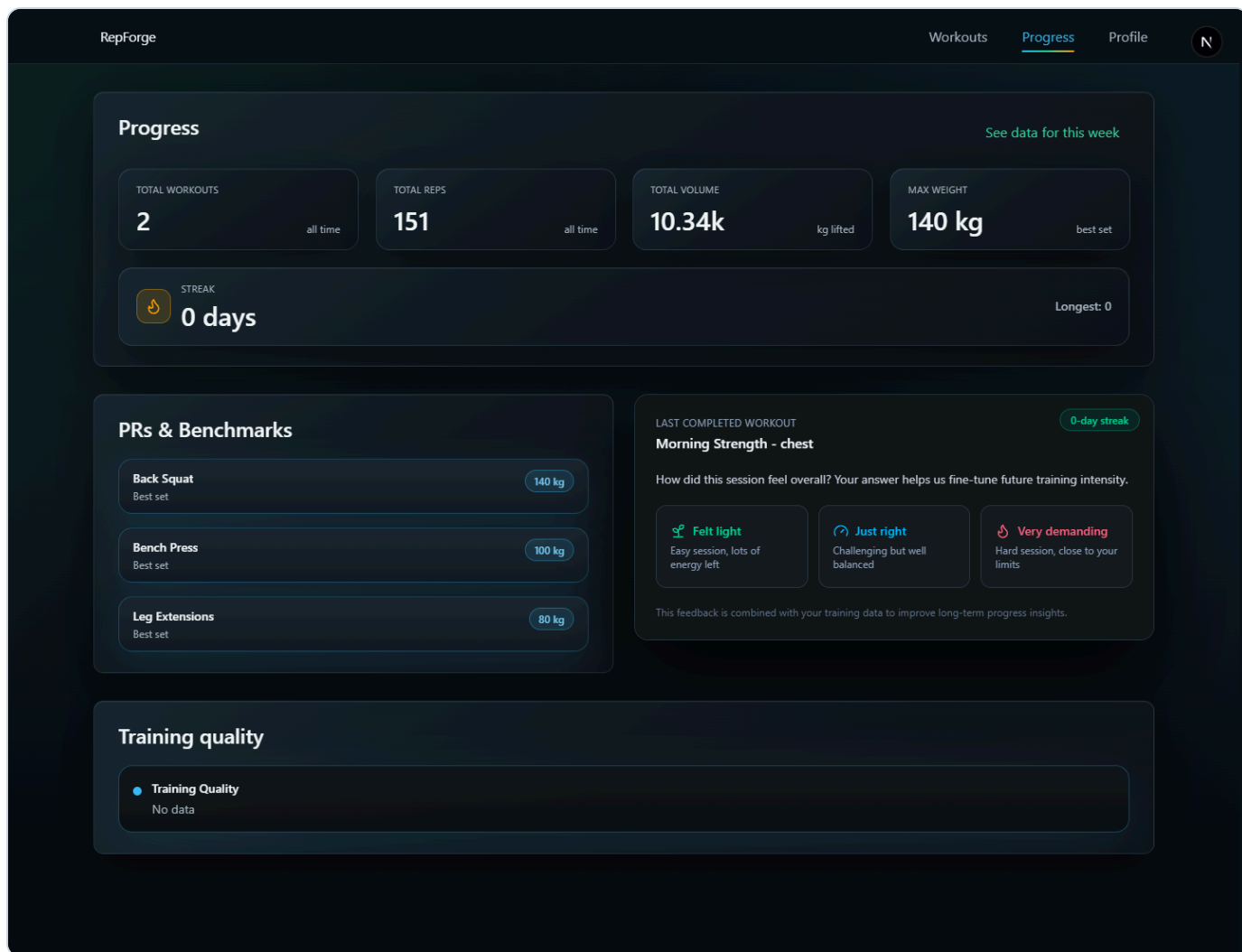
Panel analizy formy. Panel analizy formy ćwiczenia pozwala wybrać obsługiwany typ ćwiczenia, nagrać lub przesłać wideo i uruchomić analizę beta.

7. Postęp i rekordy

Progress zbiera dane z zakończonych treningów i pokazuje je w formie, która ma sens po dłuższym używaniu aplikacji. Są tu liczby ogólne, objętość, maksymalny ciężar, seria rekordów osobistych i prostsze informacje jakościowe.

Panel rekordów ma paginację, ponieważ w pełniejszej bazie ćwiczeń lista PR-ów szybko rośnie. Zamiast wciskać wszystko na raz, aplikacja pokazuje część danych i pozwala przechodzić dalej.

Pełny widok Progress pokazuje, jak statystyki i panele analityczne układają się na desktopie.



Postęp i statystyki. Ekran postępu pokazuje łączną liczbę treningów, powtórzeń, objętość, rekordy i jakość treningów.

PRs & Benchmarks jest celowo pokazane osobno, bo to przykład panelu, który musi dobrze działać przy dużej liczbie rekordów.

PRs & Benchmarks

Back Squat

Best set

140 kg

Bench Press

Best set

100 kg

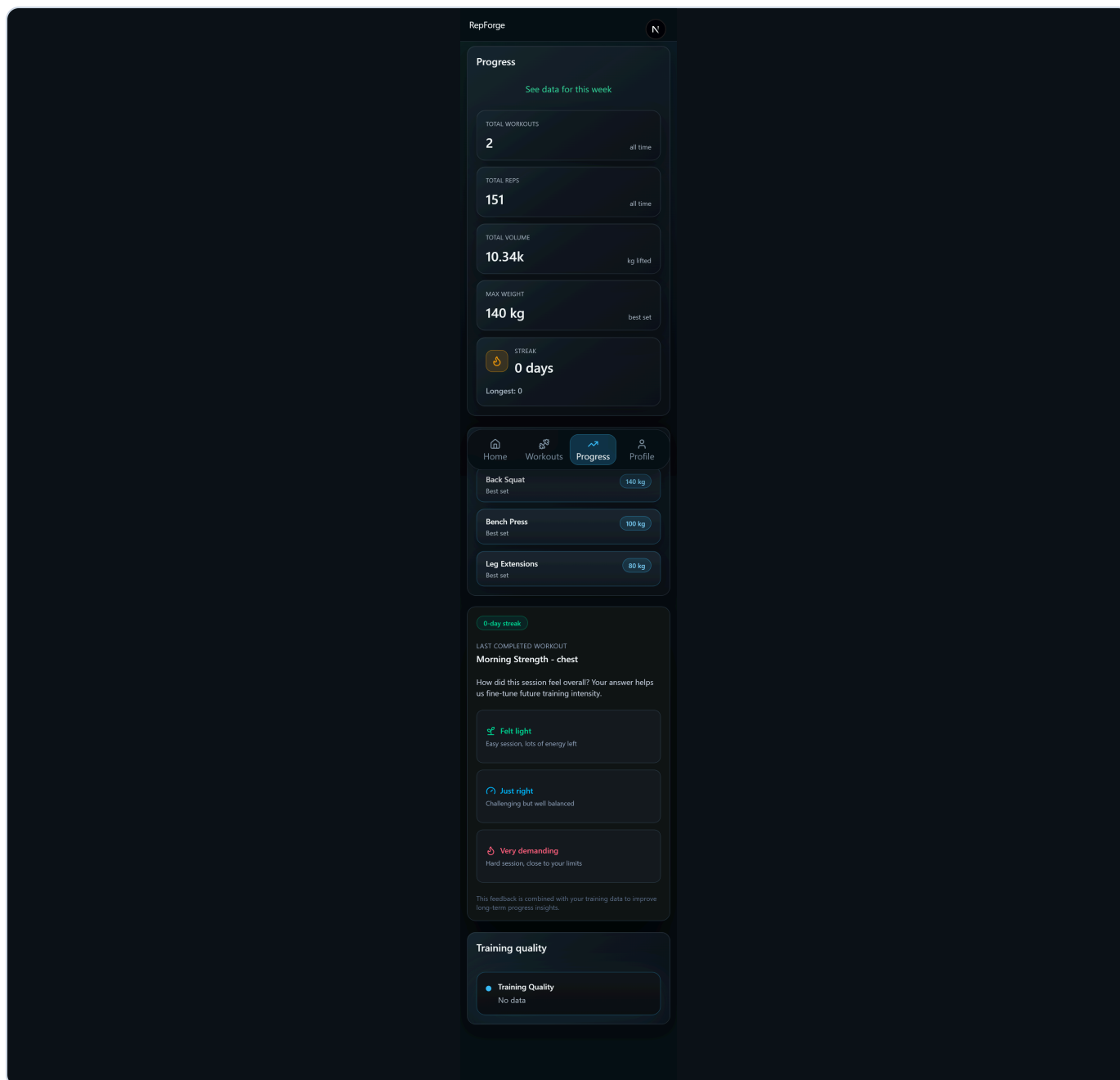
Leg Extensions

Best set

80 kg

Rekordy i paginacja. Panel rekordów osobistych prezentuje najlepsze wyniki i obsługuje większą liczbę pozycji przez paginację.

Na telefonie statystyki przechodzą w czytelny układ jednokolumnowy. To lepsze niż skalowanie desktopowego gridu na siłę.



Postęp na telefonie. Statystyki postępu przechodzą w czytelny układ jednokolumnowy na małych ekranach.

8. Profil i konto

Profil jest prosty, ale ma kilka zabezpieczeń. Dane profilu można edytować osobno, a zmiana hasła wymaga wcześniejszej weryfikacji aktualnego hasła. To drobna rzecz, ale podnosi wiarygodność aplikacji: operacje wrażliwe nie są wrzucone do jednego zwykłego formularza.

Pełny profil pokazuje trzy obszary: konto, dane profilu i zmianę hasła.

RepForge

WorkoutsProgressProfile

N

Profile

Manage account details and basic preferences.

Logout

Account

Basic authentication settings.

Login

user

Verify password

Required for sensitive actions

Current password

Verify

Profile details

Optional information (stored locally in mock mode).

Name

e.g. Szymon

Email

you@example.com

Birth date

dd/mm/yyyy

Save changes

Change password

Use the verified password above to confirm the change.

Verify your current password above to enable changes.

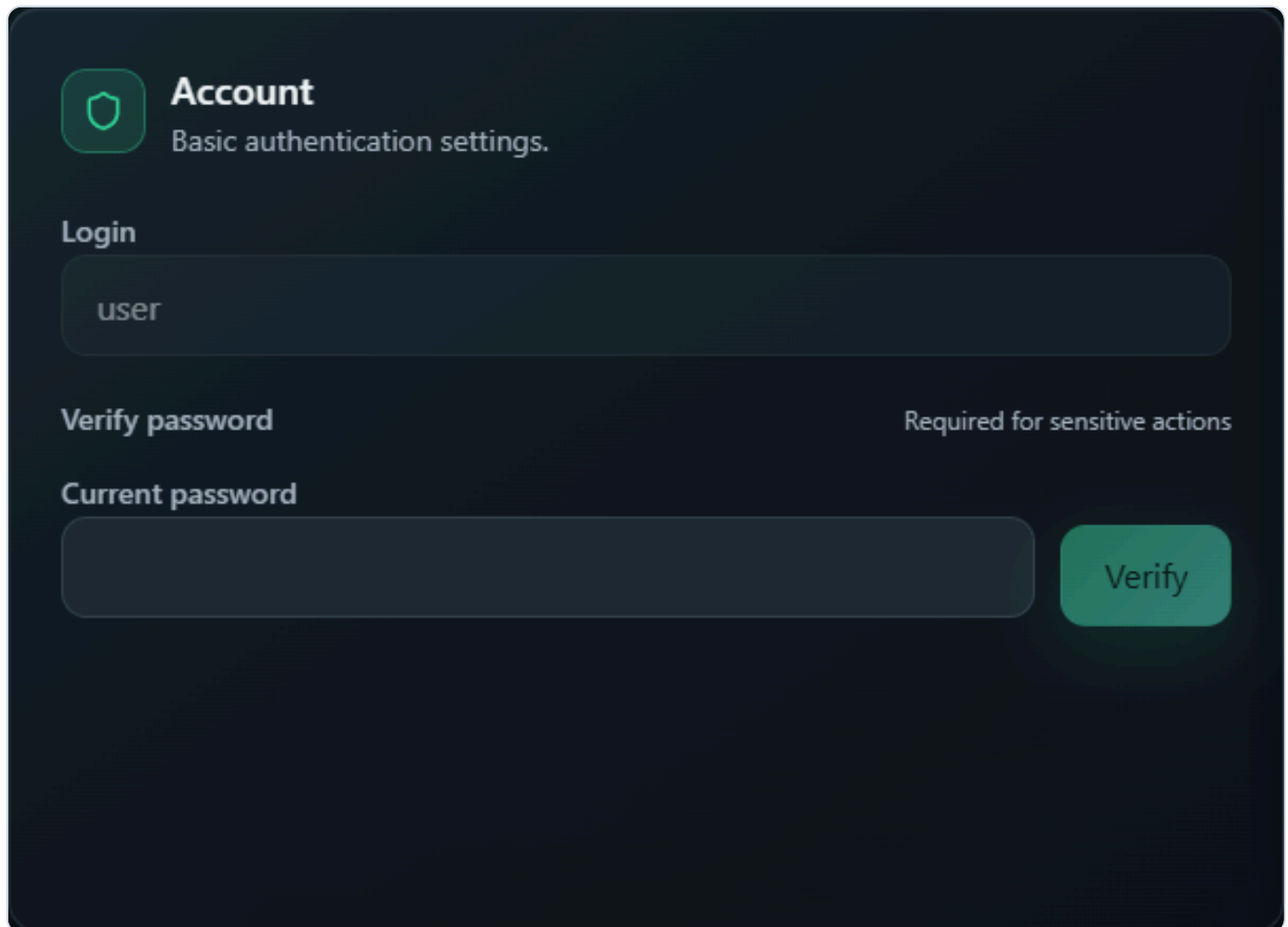
New password

Confirm new password

Change password

Profil użytkownika. Profil umożliwia zarządzanie danymi konta, szczegółami profilu, weryfikacją hasła i zmianą hasła.

Panel konta jest pierwszą bramką do operacji wrażliwych.



 **Account**
Basic authentication settings.

Login

user

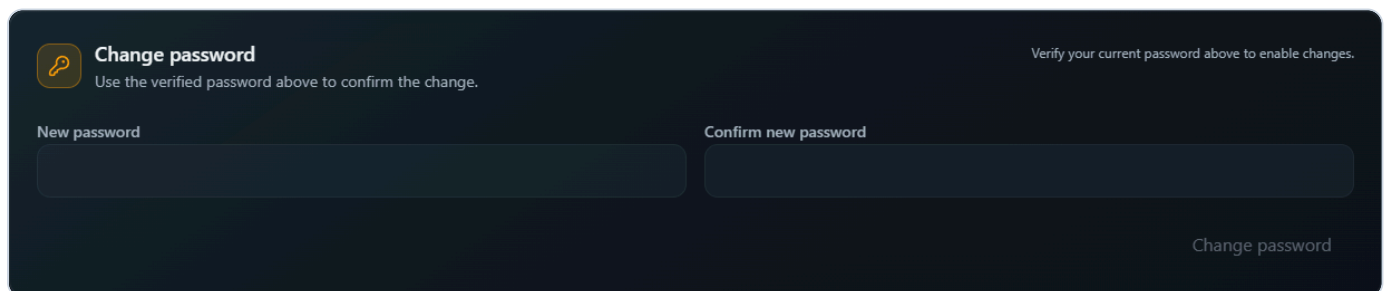
Verify password Required for sensitive actions

Current password



Weryfikacja konta. Panel konta pokazuje login i osobną weryfikację hasła, bo operacje wrażliwe nie powinny być dostępne przypadkiem.

Zmiana hasła ma własny panel, własne błędy i własny stan zapisu.



 **Change password**
Use the verified password above to confirm the change.

Verify your current password above to enable changes.

New password

Confirm new password

Change password

Zmiana hasła. Zmiana hasła jest wydzielona jako osobna sekcja i aktywuje się dopiero po poprawnej weryfikacji aktualnego hasła.

9. Architektura techniczna

Frontend

Frontend jest zbudowany w Next.js 15, React 19 i TypeScript. UI korzysta z Tailwind CSS, Framer Motion do animacji oraz TanStack Query do pobierania, cache'owania i odświeżania danych. Kod jest

podzielony na komponenty stron, view modele, hooki API, mapery i typy domenowe. Dzięki temu widok nie musi znać szczegółów backendu, a tryb mock i real mogą korzystać z podobnych kontraktów.

Backend

Backend to ASP.NET Core 8 z Entity Framework Core i PostgreSQL. API obsługuje treningi, sesje treningowe, profil, uwierzytelnianie JWT oraz trwałość analiz formy. Dla uruchamiania lokalnego i w Dockerze przygotowano konfigurację CORS, health checki oraz migracje bazy danych.

Analiza wideo

Warstwa analizy formy jest wydzielona od głównego API. Backend zapisuje żądanie i pliki, a skrypt Python z OpenCV/YOLO wykonuje analizę. Takie podejście pozwala później wymienić model albo rozbudować analizę bez przepisywania modułów treningu.

Tryby danych i uruchamiania

Tryb	Opis	Zastosowanie
mock	Dane lokalne w przeglądarce, bez backendu.	Demo, szybkie testy UI, prezentacja bez Dockera.
real	ASP.NET Core API + PostgreSQL.	Pełny przepływ z bazą danych, JWT i trwałością danych.

Najważniejsze komendy

- Web PC mock: `.\scripts\run-frontend-local.ps1 -Mode mock`
- Web PC real: `docker compose --profile real up -d postgres backend` oraz `.\scripts\run-frontend-local.ps1 -Mode real -ApiUrl http://localhost:5064/api`
- Web mobile: frontend uruchomiony z `-Mobile` i adresem IP komputera.
- Android app: Capacitor łąduje działający frontend przez `CAPACITOR_SERVER_URL`.

Dlaczego dwa tryby? Mock jest dobry do szybkiej prezentacji i pracy nad UI. Real API jest potrzebne do pokazania trwałości danych, JWT, migracji, health checków i integracji z PostgreSQL. Oba tryby są celowo utrzymywane, bo rozwiązują inne problemy.

10. Obsługa błędów i ciężkich przypadków

Duża część pracy przy aplikacji polegała na domknięciu zachowań, które zwykle wychodzą dopiero podczas ręcznego klikania. Chodziło o to, żeby użytkownik nie zobaczył surowego `Request failed`,

żeby timer nie gubił sesji po przejściu między zakładkami i żeby mobile nie miał elementów nachodzących na systemowe przyciski.

- Brak dostępu do API jest mapowany na przyjazny komunikat użytkownika, a nie surowy HTTP 500.
- Formularze treningu i profilu blokują zapis, gdy dane są niepoprawne.
- Usunięcie treningu wymaga potwierdzenia.
- Zakończenie sesji wymaga potwierdzenia w dialogu.
- Timer treningu może przekroczyć zero i nie zmienia etapu bez akcji użytkownika.
- Aktywna sesja może być kontynuowana po opuszczeniu widoku treningu.
- Tryb mobile uwzględnia dolną nawigację, safe area i pływające elementy przeglądarki.
- Analiza wideo ma limity rozmiaru, retencji i liczby analiz, a niepowodzenie jest obsługiwane kontrolowanym stanem beta.
- W trybie real API wymagane jest poprawne CORS i silny sekret JWT poza środowiskiem Development.

11. Podsumowanie

RepForge domyka pełny przepływ osoby trenującej: planowanie, wykonanie sesji, zapis historii, przegląd postępu, konto użytkownika i eksperymentalną analizę techniki z wideo. Aplikacja działa jako web PC, web mobile oraz Android shell. Może działać szybko w mocku albo pełniej w trybie real z backendem, PostgreSQL i trwałością danych.

Najważniejsze w projekcie nie jest samo dodanie wielu ekranów, tylko połączenie ich w jeden sensowny rytm: użytkownik widzi plan, robi trening, nie traci sesji po przejściu między widokami, a potem ma dane do sprawdzenia postępu. To sprawia, że aplikacja jest gotowa do pokazania jako spójny produkt, a nie tylko zbiór osobnych formularzy.